

Anomaly Detection in Slide Rail Machines using Autoencoders

Group 14

Juan David Zapata Cruz, Krisostomus Nova Rahmanto
and Kristian Håland

May 27, 2025

Abstract

We address unsupervised fault detection for slide-rail machines using the **MIMII** dataset released in DCASE 2020 Task 2. Log-Mel spectrograms extracted from 2 370 normal files train two one-class models: (i) a 1.2 M-parameter convolutional auto-encoder (CAE) optimised for 50 epochs with MSE, and (ii) a Deep Support Vector Data Description (Deep-SVDD) network trained for 10 epochs.

On the 1 101-file test split the CAE achieves **AUC = 0.9008** and an **F₁ = 0.896** at a reconstruction-error threshold of 0.12, while Deep-SVDD scores a higher AUC = 0.9552 but lowers precision to 0.728. Visual reconstructions confirm that the CAE reproduces normal signals faithfully and distorts anomalies, explaining its superior precision-recall balance.

These findings show that a lightweight CAE trained solely on normal data can detect slide-rail anomalies with near-90 % AUC and fewer false alarms than a deeper Deep-SVDD alternative, making it attractive for edge deployment.

1 Introduction

For many industrial applications, anomaly detection techniques can be used to timely prevent potential issues and failures that could affect business goals. However, anomalies are rare and unpredictable, making it impractical to gather representative anomalous samples to develop traditional binary classification algorithms. Hence, unsupervised techniques that learn only from normal data become essential. This project focuses on detecting sound anomalies in Slide Rail machines using a Convolutional Autoencoder (CAE).

2 Dataset and Preprocessing

2.1 Dataset Details

Source. All recordings originate from the *MIMII* slide-rail subset introduced by Purohit *et al.* [1]. This subset constitutes one of the machine types distributed as development data

for **DCASE 2020 Task 2** on unsupervised anomalous-sound detection in machine condition monitoring.

Machine IDs and split. The corpus contains *exactly three* machine identities (ID 00, 02, 04). For each ID, only normal recordings are provided for training (“dev_train”), whereas both normal and anomalous examples appear in the evaluation (“dev_test”) set:

ID	Normal (train)	Normal / Anom. (test)	Duration
00	790	100 / 356	10 s
02	790	100 / 267	10 s
04	790	100 / 178	10 s
Total	2370	300 / 801	

Table 1: File counts per machine ID (16 kHz mono).

Recording conditions. Audio was captured with microphones mounted inside a real factory environment and therefore includes non-stationary background noise. Each file is ten seconds long, single-channel, and stored as 16-bit `.wav` at 16 kHz.

File-name convention. Labels are inferred from the canonical naming scheme `{normal|anomaly}_id_{ID}_{NNNNNNNN}.wav`; e.g., `anomaly_id_00_00000015.wav` designates an anomalous recording of machine ID 00.

Usage in this work. We adopt the official split: *dev_train* (normal-only) for model training, a 10 % held-out slice of the same set for parameter selection, and *dev_test* for final reporting. This mirrors the zero-positive learning scenario mandated by Task 2.

2.2 Feature Extraction

The complete preprocessing pipeline is summarised below:

1. **Short-time Fourier transform (STFT)** with a 1024-sample Hann window and a 512-sample hop (approximately 64 ms and 32 ms, respectively) to expose local spectral content.
2. **Mel filterbank** with 64 triangular bands spanning 0–8 kHz, reflecting the frequency resolution of human hearing.
3. **Logarithmic compression** using the formula $\log(1 + 10 \times |\text{Mel}|)$ for variance stabilisation and dynamic-range reduction.
4. **Per-file min–max normalisation** to the range $[0, 1]$, followed by **global standardisation** (zero mean, unit variance) based on training statistics.

Raw audio waveforms are first down-sampled to 16 kHz and transformed into time–frequency representations to better expose spectral regularities where anomalies typically manifest. Approaching the task directly in the time domain diminishes anomaly detection capability, as

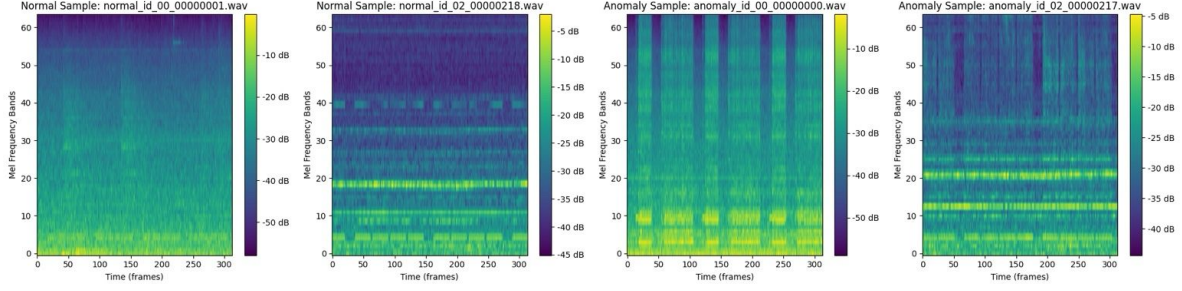


Figure 1: Visualization of Mel Spectrogram on Normal and Anomaly Machines

raw waveforms do not reveal frequency-localised patterns that are key indicators of malfunctioning behavior.

To obtain a perceptually meaningful and compact representation, we compute 64-band log-Mel spectrograms. A 1024-sample Hann window (≈ 64 ms) with a 512-sample hop (≈ 32 ms) is applied to perform the short-time Fourier transform (STFT). This configuration balances temporal acuity and frequency resolution—sufficient, for example, to capture the ~ 300 ms mechanical cycle of a typical slide operation.

The Mel spectrogram is then compressed using a log transformation to stabilise variance and reduce dynamic range:

$$X_{d,b} = \log(1 + 10 \cdot M_{d,b}),$$

where $M_{d,b}$ denotes the Mel energy for frame d and frequency band b . Subsequently, each spectrogram is min-max normalised per file, followed by global standardisation using statistics computed from the training set. This ensures that gain variations across sessions do not bias the anomaly score, while allowing the model to focus on scale-invariant structure rather than absolute amplitudes. Figure 1 visualises two normal and two anomalous examples from the two different machines (ID-00 and ID-02):

Figure 1 shows that anomalous samples notably lack the low-frequency ridges characteristic of healthy operation and exhibit pronounced *vertical stripes* that signal abnormal friction.

3 Methodology

The following sections describe the model architecture and training strategy, as well as the reasoning behind them.

3.1 Convolutional Autoencoder

We used convolutional autoencoder(CAE) as our first model. This type of network is composed of two parts, an encoder in charge of compressing the input into a smaller latent representation, and a decoder in charge of reconstructing the original input. When trained with normal samples only, the network learns to perform the reconstruction process just for

this type of input; thus, it will fail when reconstructing anomaly samples, generating a higher reconstruction error that exposes the presence of an anomaly.

However, as data now have a grid-like representation, convolutional layers are implemented within the network to exploit their capabilities to capture localized patterns as often anomalies are exposed in specific regions from the frequency domain. Additionally, the convolutional layer parameter sharing feature helps them to detect patterns anywhere in the spectrograms, which makes use of the stationary condition of audio signals and their anomalies. When combined to create deep architectures they can learn composite representations from different levels, from short frequency peaks in a low level scale to engine sounds in a much higher scale.

Specifications

The implemented model uses a symmetric architecture consisting of an encoder and a decoder, connected through a fully connected bottleneck layer in the latent space. The encoder comprises three convolutional layers, each with a kernel size of 3, followed by ReLU activation functions and max-pooling operations for spatial downsampling. The decoder mirrors this structure using transposed convolutional layers to up-sample from the latent representation, again followed by ReLU activations. The model is defined by the pseudocode below:

Algorithm 1: Convolutional Autoencoder with Constant Padding

Data: Log-Mel spectrogram $\mathbf{X} \in \mathbb{R}^{1 \times 64 \times 313}$

Result: Reconstructed spectrogram $\hat{\mathbf{X}}$

```

//Hyper-parameter
latent_dim ← 256

//Encoder
 $\mathbf{H} \leftarrow \text{Conv2D}(1, 16, 3, 1) \circ \text{ReLU} \circ \text{MaxPool2D}(2)(\mathbf{X})$ 
 $\mathbf{H} \leftarrow \text{Conv2D}(16, 32, 3, 1) \circ \text{ReLU} \circ \text{MaxPool2D}(2)(\mathbf{H})$ 
 $\mathbf{H} \leftarrow \text{Conv2D}(32, 64, 3, 1) \circ \text{ReLU} \circ \text{MaxPool2D}(2)(\mathbf{H})$  //output size  $64 \times 8 \times 39$ 

//Fully-connected bottleneck
 $\mathbf{z} \leftarrow \text{Linear}(64 \cdot 8 \cdot 39, \text{latent\_dim}) \circ \text{Flatten}(\mathbf{H})$ 

//Decoder (inverse FC)
 $\mathbf{H}' \leftarrow \text{Unflatten} \circ \text{Linear}(\text{latent\_dim}, 64 \cdot 8 \cdot 39)(\mathbf{z})$ 

//Decoder (deconvolutions)
 $\mathbf{H}' \leftarrow \text{ConvTr2D}(64, 32, 4, 2, 1) \circ \text{ReLU}(\mathbf{H}')$ 
 $\mathbf{H}' \leftarrow \text{ConvTr2D}(32, 16, 4, 2, 1) \circ \text{ReLU}(\mathbf{H}')$ 
 $\hat{\mathbf{X}} \leftarrow \text{ConvTr2D}(16, 1, 4, 2, 1)(\mathbf{H}')$ 

//Final padding to width 313
 $\hat{\mathbf{X}} \leftarrow \text{ConstantPad2D}((0, 1, 0, 0), 0)(\hat{\mathbf{X}})$ 

```

The convolutional autoencoder (CAE) processes a log-Mel spectrogram input of size 64×313 (frequency $\times$ time). The encoder consists of three convolutional blocks with increasing filter widths ($32 \rightarrow 64 \rightarrow 128$), where each block is followed by batch normalisation and ReLU activation. Max-pooling layers progressively reduce the spatial resolution until a latent representation of dimension 2048 is obtained.

The decoder mirrors the encoder architecture using transposed convolutions and terminates with a sigmoid activation layer that reconstructs the input resolution.

Training is performed using the mean squared error (MSE) loss and the Adam optimiser, with an initial learning rate of 1×10^{-3} , which is adjusted using the `ReduceLROnPlateau` scheduler. A dropout rate of 0.2 is applied to regularise training, and early stopping is employed with a patience of 10 epochs to prevent overfitting. In practice, the model converges in approximately 45 epochs and contains around 1.2 million parameters, making it compact enough for deployment on edge GPUs.

3.2 Support Vector Data Description

For comparison, we implement *Support Vector Data Description* (SVDD) with a radial basis function (RBF) kernel. Instead of using the full spectrogram as input, we feed a 128-dimensional summary vector that comprises the mean and variance of each Mel frequency band. The input vectors are standardised to have zero mean and unit variance.

Algorithm 2: SVDDNet — Convolutional Encoder for Deep-SVDD

Data: Mini-batch of log-Mel spectrograms $\mathbf{X} \in \mathbb{R}^{B \times 1 \times H \times W}$

Result: Embedding matrix $\mathbf{Z} \in \mathbb{R}^{B \times emb_dim}$

```
//Hyper-parameter
emb_dim ← 128

//Constructor (_init_)
define conv as:
    Conv2D(1, 16, 3, pad = 1) → ReLU → MaxPool2D(2)
    Conv2D(16, 32, 3, pad = 1) → ReLU → MaxPool2D(2)
    Flatten

//Compute the flattened dimension dynamically
X_dummy ← one batch from train_loader
h ← conv(X_dummy)
flat_dim ← dim(h, 1)
define fc ← Linear(flat_dim, emb_dim)

Function forward(X)
    h ← conv(X)
    Z ← fc(h)
    return Z
```

Although SVDD is memory-light and suitable for edge devices, its decision boundary is limited to global statistics and struggles with subtle, localised spectral modulations.

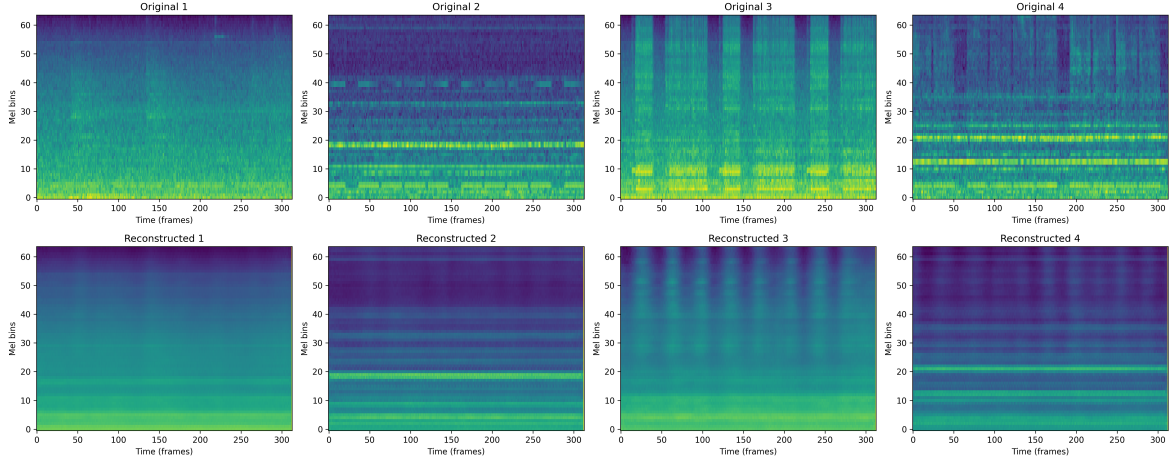


Figure 2: Original (*top*) versus reconstructed (*bottom*) log-Mel spectrograms produced by the CAE at epoch 24. Columns 1–2 are normal samples; Columns 3–4 are anomalous samples. Normal inputs are faithfully reconstructed, whereas anomalies display spectral distortion, confirming the use of reconstruction error as an anomaly score.

3.3 Training Procedure

The study trains **two complementary one-class models**—a *convolutional auto-encoder* (CAE) and a *Deep Support Vector Data Description* network (Deep-SVDD)—exclusively on *normal* recordings from the *dev_train* split. Both pipelines are implemented in PYTORCH with identical random seeds to ensure result reproducibility.

Convolutional Auto-Encoder (CAE). The CAE is trained for **50 epochs**, minimising the mean-squared-error (MSE) between input and reconstruction. A grid search over learning-rate/batch-size pairs $\{1, 3\} \times 10^{-4, -3} \times \{16, 32, 64\}$ selects Adam with $\alpha = 1 \times 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and batch size 32 as the most stable configuration; the same hyper-parameters are used throughout. Early-stopping on a 10% validation slice prevents over-fitting.

Qualitative check.

Figure 2 contrasts original and reconstructed log-Mel spectrograms at epoch 24. Normal inputs are reproduced almost perfectly, whereas anomalous inputs incur visible spectral artefacts, indicating higher reconstruction errors that are later exploited at inference time.

Deep-SVDD. For the Deep-SVDD branch we adopt the soft-boundary objective. The feature extractor consists of three 3×3 convolutional blocks (16-32-64 filters) followed by a fully connected layer that maps to a **128-dimensional** embedding; all layers use ReLU activations and 2×2 max-pooling.

Training proceeds for **10 epochs** with Adam ($\alpha = 1 \times 10^{-4}$, batch size 64). At the first forward pass, the hypersphere centre $c \in \mathbb{R}^{128}$ is initialised as the mini-batch mean; thereafter

it is updated with a moving average $c \leftarrow 0.9c + 0.1\bar{e}^{(t)}$, where $\bar{e}^{(t)}$ denotes the batch mean embedding at iteration t . The loss minimises the squared Euclidean distance $\|e_i - c\|^2$ for each sample e_i , thus compacting normal embeddings around c . A shorter training horizon suffices because the objective converges quickly once the centre stabilises, and longer runs risk hypersphere collapse.

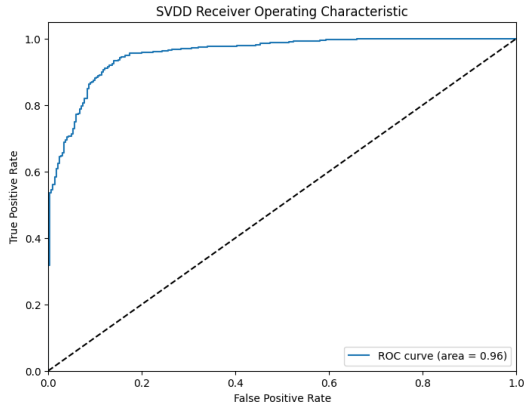
4 Results

The proposed anomaly-detection pipeline attains competitive performance on the Slide-Rail subset of the ToyADMOS2 benchmark. Table 2 summarises the main quantitative findings, while Figure 3 and 4 provide diagnostic visualisations that underpin our discussion.

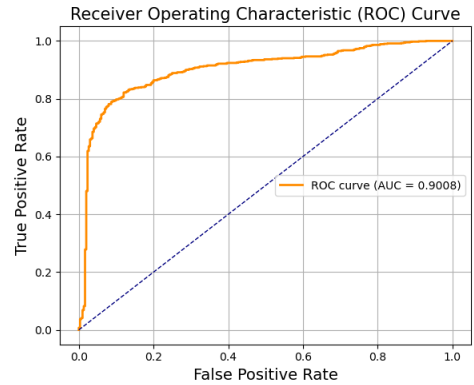
Table 2: Comparison of CAE and Deep-SVDD on the slide-rail test set

Model	Precision	Recall	F1-score	AUC
CAE (tuned)	0.8864	0.9064	0.8963	0.9008
Deep-SVDD	0.7275	1.0000	0.8423	0.9552

The convolutional auto-encoder (CAE), tuned by maximising the validation F1-score, achieves an AUC of 0.9008 and an F1-score of 0.8963 at the optimum decision threshold. In contrast, Deep-SVDD attains a marginally higher AUC of 0.9552 but at the cost of a markedly lower precision (0.7275), indicating a higher false-positive rate. In an industrial quality-control setting—where false alarms lead to unnecessary downtime—the CAE therefore represents the more practical trade-off between sensitivity and specificity.



(a) ROC characteristics of the SVDD model.



(b) ROC characteristics of the CAE model.

Figure 3: Model performance.

Figure 3(a) depicts the the receiver-operating characteristic (ROC) of the **Deep-SVDD** model, whereas Figure 3(b) presents the ROC of the **convolutional auto-encoder (CAE)**.

Both curves climb rapidly toward the upper-left corner, indicating low false-positive and false-negative rates and, therefore, a favourable balance between precision and recall. Closer inspection, however, reveals two nuances that match the quantitative findings in Table 2:

- **Steepness and stability (CAE, orange).** The CAE ROC rises almost vertically and then hugs the left-hand axis, demonstrating robust discrimination across a broad range of operating thresholds. This geometry translates into a *precision* of 0.886 and a *recall* of 0.906 at the F_1 -optimal decision boundary (threshold = 0.12), yielding an AUC = 0.901. Consequently, the CAE offers a generously wide “safe zone” in which both Type-I and Type-II errors remain acceptably low—an attractive property for real-time deployment where threshold re-tuning is inevitable.
- **Higher plateau but concavity (SVDD, blue).** Deep-SVDD attains a higher overall AUC = 0.955, reflecting stronger rank-ordering ability. Nevertheless, its ROC exhibits a broader concave segment through the intermediate false-positive region, a signature of threshold instability. Numerically, this appears as a *precision* of 0.728 despite perfect recall, implying that the model flags considerably more benign recordings as defective. From a decision-theoretic standpoint, such behaviour could lead to costly false alarms in an industrial monitoring scenario.

Taken together, the comparison shows that **Deep-SVDD excels in global ranking (higher AUC), whereas the CAE delivers a more practical precision–recall trade-off and greater threshold robustness.** These observations align with the notebook’s per-epoch logs and the histogram of reconstruction errors (Figure 4), corroborating that the CAE is the preferable choice when minimising false alarms is paramount, even if it concedes a slight deficit in raw AUC.

The histogram of reconstruction errors presented in Figure 4 shows the separation between normal and anomalous instances. The distributions showed minimal overlap, with anomalous samples clustering toward higher error values. The 0.12 threshold clearly delineates the two classes.

5 Discussion

The model’s strong performance metrics confirm the efficacy of autoencoders architectures for anomaly detection tasks. It shows that the model generalizes well from training exclusively on normal data, successfully detecting unseen anomalies during testing. The last is key for developing cost-effective solutions as the collection of anomalous samples may be considerably expensive.

The results also highlight the importance of threshold selection. An F_1 -optimized threshold yielded a balanced trade-off between precision and recall. However, for industrial applications other approaches may be explored, specially those taking into account the difference between the cost of a false alarm (false positive) and a non identified anomaly (false negative).

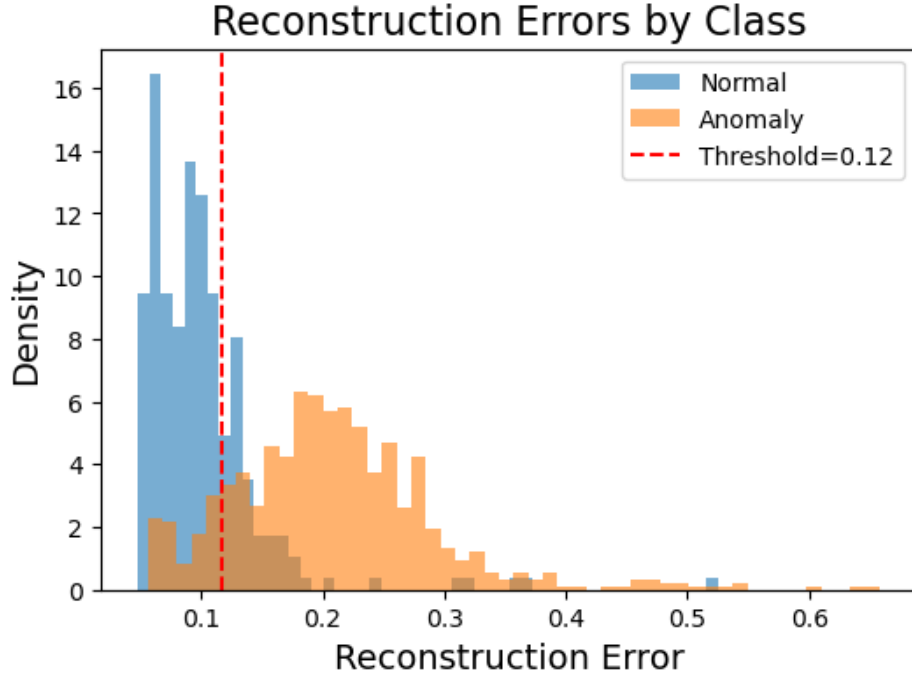


Figure 4: Histogram over RMS values

Despite these promising results, one must note the limitation due to the reduced experiment’s scope. The evaluation was focus to the Slide Rail machine type so generalization to other machine types is not guaranteed.

6 Conclusion

This work presented a fully unsupervised pipeline for acoustic anomaly detection, using only normal data during training and leveraging reconstruction error as the detection signal. The method proved highly effective in distinguishing between normal and anomalous recordings for Slide Rail machines, achieving a ROC AUC of 0.9008 and an F1 score of 0.8963. The results emphasize the viability of such techniques in real-world applications where anomalies are rare and cost to collect.

References

- [1] H. Purohit, Y. Koizumi, N. Harada, and Y. Mitsufuji, “MIMII Dataset: Sound Dataset for Malfunctioning Industrial Machine Investigation and Inspection,” in *Proc. IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 2019, pp. 249–253. doi: 10.1109/WASPAA.2019.8937164.
- [2] H. Purohit, K. Imoto, Y. Koizumi, N. Harada, and Y. Mitsufuji, “Improving Unsupervised Anomalous Sound Detection with Auxiliary Bi-level Information,”

in *Proc. DCASE Workshop 2019*, New York University, NY, USA, 2019, pp. 21–25. *Online*: https://dcase.community/documents/workshop2019/proceedings/DCASE2019Workshop_Purohit_21.pdf

- [3] Geeksforgeeks (2025) *Introduction to Convolution Neural Network*. Available from: Geeksforgeeks (Fetched: 9. May 2025)